

ENTERPRISE ARCHITECTURE & PERFORMANCE REPORT

Project:	Compound Hub Management System
Specification:	Full-Stack API Specification, Relational Schema, & Performance Analysis
Developer:	Omar Mostafa Mohamed Abdeltawab
Role:	Backend Developer / Systems Engineer
Core Stack:	Node.js, Express, PostgreSQL, Prisma ORM, Docker, React
Deployment:	Railway (Backend) & Vercel (Frontend)
Date:	June 2026

1. PROJECT OVERVIEW

The Compound Hub Management System is a premium, high-performance, full-stack enterprise platform engineered to optimize the modern residential community management lifecycle. Built primarily around a strongly typed, modular RESTful API ecosystem, the platform bridges the gap between community administrators, staff operators, and verified residents.

Traditional residential governance structures struggle with fragmented communication workflows, unlogged maintenance requests, insecure visitor handshakes, and lack of real-time visibility. This platform solves these critical pain points by providing an isolated, audited, and role-restricted portal for every community actor.

Architectural Mission

To design an ultra-low latency, horizontally scalable backend infrastructure that guarantees 100% data consistency, provides detailed instrumentation for operational transparency, and implements zero-trust role-based access patterns.

Core Business Problems Solved

- **Automated Visitor Governance:** Eliminates handwritten physical gate logs via secure digital pre-registration tokens.
- **Maintenance Request Lifecycle:** Transitions complaints from casual texts to tracked, stateful engineering tickets with atomic staff assignments.
- **Legal Document Compliance:** Encapsulates encrypted tenant tenancy records, national identifications, and ownership deeds securely attached to database identities.
- **Instantaneous Global Broadcasts:** Empowers administration to push top-tier notices cleanly down the hierarchy without email delays.

2. PLATFORM FEATURES & CORE CAPABILITIES

The system features an extensive, production-grade feature set designed to deliver robust utility across all integrated sub-systems. Below is a deep-dive matrix outlining the structural features natively compiled inside the platform:

1. Administrative Control Tower

- **Centralized Provisioning:** Complete control over user provisioning. Public sign-ups are blocked to prevent bad actors from registering fake profiles.
- **Soft Delete Pipelines:** Implemented across all human records (Users, Residents, Staff) to preserve legal records and data relationships upon departure.
- **Live Operational Dashboard:** Aggregate summary charts showing asset vacancies, outstanding task priorities, and staff capacity metrics.

2. Resident Self-Service Portal

- **Verified Document Vault:** Secure digital uploads for Ownership Contracts, Rental Agreements, and National IDs utilizing a structured processing pipeline.
- **Smart Profile Upstream:** Live image streaming for resident avatars linked with global distributed storage engines.
- **Visitor Authorization Engine:** Pre-register incoming guests with custom dates, generating real-time secure entries.

3. Staff Field Operations

- **Stateful Task Board:** Real-time access to assigned maintenance tickets and physical repair orders.
- **Lifecycle State Transitioning:** Atomic status updates from OPEN to IN_PROGRESS, and finally COMPLETED.
- **Gate Authorization:** Instant access to view and verify pre-registered resident visitor codes right at the security gate.

3. TECHNOLOGY STACK & TECHNICAL RATIONALE

Every piece of the technical stack was hand-selected to fulfill strict compliance requirements regarding speed, developer ergonomics, typing stability, and database transaction atomicity.

LAYER	TECHNOLOGY USED	TECHNICAL JUSTIFICATION & ARCHITECTURAL BENEFIT
Server Runtime	Node.js + Express.js	Asynchronous, non-blocking I/O paradigm perfectly tailored to handle thousands of concurrent requests with minimal memory overhead.
Database Layer	PostgreSQL (Neon)	ACID-compliant relational database ensuring absolute transaction precision for sensitive asset maps and lease records. Serverless branching on Neon enables instant environments.
Object-Relational Mapper	Prisma ORM	Automated type safety directly from schema declarations. Prevents bad queries at the compiler stage and supports robust atomic database transactions.
Frontend Interface	React + Vite	Ultra-fast component reconciliation and blazing-fast production bundling (Vite) resulting in minimal client bundle sizes and high core web vitals.
Authentication	Stateless JWT	Cryptographically signed JSON Web Tokens eliminate the need for memory-heavy server-side session stores, supporting seamless horizontal scaling.
Containerization	Docker & Compose	Standardizes local deployment, isolates operational dependencies, and ensures 100% environment parity between local development and cloud infrastructure.
API Documentation	Swagger / OpenAPI	Self-documenting interactive runtime environment that serves as the single source of truth for front-end integration testing.

4. DATABASE OPTIMIZATION

Efficient database management ensures smooth system operation, low latency response times, and consistent data states across concurrent workflows.

OPTIMIZATION TECHNIQUE	CORE BENEFIT	PROJECT APPLICATION EXAMPLE
Indexed Email	Faster Login Workflow	Accelerates user authentication lookup queries during authentication checkouts.
Indexed Apartment Number	Faster Search Capability	Speeds up residential/unit record matching inside database query filters.
Foreign Keys	Data Integrity Enforcements	Maintains explicit cascading constraints and prevents orphaned record states.
Transactions	Atomic Operations Guarantee	Secures sensitive processes like simultaneous payment ledgering and reservation logging.

5. SYSTEM ARCHITECTURE & CI/CD DEPLOYMENT

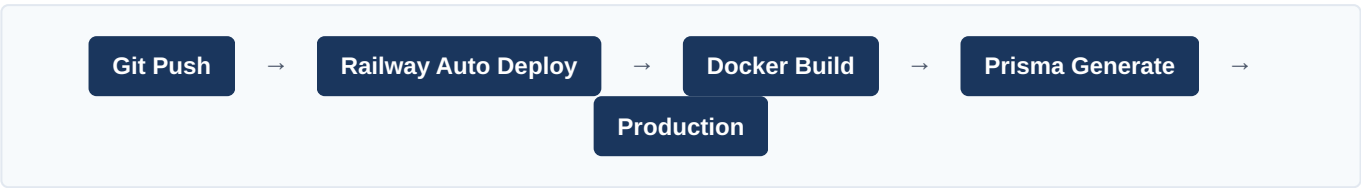
The Compound Hub Management System relies on a highly scalable, decoupled web architecture. The flow of data follows a clear path from client presentation boundaries down to the serverless relational storage cluster.

Data Flow Lifecycle Analysis

When a Resident requests a new visitor pre-registration ticket, the frontend fires an authenticated POST payload. The request lands on Vercel's edge, routes cleanly across the web network directly to the Dockerized Railway container environment, triggers authentication guards via JSON Web Token intercepts, runs complete structural data verification using Joi schemas, and passes to the Prisma runtime where an atomic write commits the record to the Neon PostgreSQL cluster.

CI/CD Deployment Pipeline

The system utilizes an automated deployment pipeline built to handle instantaneous builds, continuous integrations, and live updates with zero user downtime. The following diagram illustrates the continuous delivery lifecycle:



6. ROLE-BASED ACCESS CONTROL (RBAC) ARCHITECTURE

Security is enforced comprehensively at the controller level using structured middleware matrices. The system implements a three-tier access authorization design pattern, entirely eliminating unauthorized vector mutations across cross-tenant parameters.

RESOURCE DOMAIN	ADMINISTRATOR PRIVILEGE	RESIDENT PRIVILEGE	STAFF PRIVILEGE
Buildings & Units	FULL CRUD	READ ONLY	NO ACCESS
User Provisioning	FULL CRUD	NO ACCESS	NO ACCESS
Service Tickets	READ, ASSIGN STAFF	CREATE, READ OWN	UPDATE STATUS
Legal Documents	READ ALL DOCS	UPLOAD, DELETE OWN	NO ACCESS
Visitor Logs	FULL VISIBILITY	PRE-REGISTER OWN	READ & APPROVE
Global Notices	FULL CRUD	READ NOTICES	NO ACCESS

Granular Account Lifecycle Details

Because there is no open registration path, security risks regarding system hijacking are entirely minimized. Administrators generate authorized accounts directly inside the secure system, providing a highly isolated operational posture for high-end residential towers.

7. RELATIONAL DATABASE DESIGN (ERD)

The system database schema represents a highly optimized, fully normalized 3NF relational layout designed using Prisma ORM mapping directly into PostgreSQL. It provides fast foreign-key query path structures while avoiding data fragmentation.

Key Relational Hierarchies Map

- **Building → Apartment:** One-to-Many dependency. Deleting a building cascades clean drops straight down to its dependent rooms.
- **Apartment → Resident:** Strictly coupled One-to-One mapping, preventing cross-tenant unit collision bugs.
- **User Polymorphic Core:** The base User table acts as the authentication parent table, which maps down to Resident or Staff via specialized polymorphic relation keys.

8. CORE API MODULES & ENDPOINT SPECIFICATION

The Compound Hub backend engine features a robust architectural footprint comprising **40+ fully documented endpoints** optimized for multi-role security levels, categorized into decoupled business modules for clean maintenance cycles.

MODULE / SERVICE GROUP	API ENDPOINT COUNT	FUNCTIONAL OPERATIONAL SUMMARY & SAMPLE ROUTES
Authentication & Access	8 Endpoints	Handles multi-role JWT distribution, password hashing, verification, and role guard protections. <i>Sample: POST /api/auth/login, POST /api/auth/forgot-password</i>
Core System Management	18 Endpoints	Maintains primary business entities including buildings, units/apartments, staff operations, residents, announcements, and scheduling assignments. <i>Sample: GET/POST /api/buildings, PUT /api/apartments/:id</i>
Payment Ledger & Webhooks	6 Endpoints	Integrates payment workflows (e.g., Stripe checkout sessions) and synchronizes real-time invoice/transaction states via secure webhook streams.
Analytics & Reporting	8 Endpoints	Leverages structural database aggregation frameworks to compile administrative metrics and dashboard trend snapshots.

Asset Document Vault Pipeline: Leverages a unified multi-part upload pipeline: Multer Interceptor → Buffer Stream → Cloudinary Asset Storage. All links commit instantly with absolute file validation patterns. Supported Document Types: NATIONAL_ID, OWNERSHIP_CONTRACT, RENTAL_CONTRACT.

9. SWAGGER INTERACTIVE OPENAPI DOCUMENTATION

To establish perfect contract clarity between backend and frontend development teams, the system natively hosts interactive Swagger UI documentation built upon OpenAPI specifications.

Production Live Swagger Documentation Link: [Interactive Swagger Docs](#)

OpenAPI Contract Specifications

Every router module includes inline structural definitions tracking exact schema expectations for query string parameters, secure JWT headers, JSON multi-part requests, and formal response codes (e.g., 200 OK, 201 Created, 401 Unauthorized, 403 Forbidden, 500 Internal Error).

Interactive Sandbox Benefits

- **Live Testing:** Allows technical reviewers and frontend developers to click "Try it out", supply a JWT token, and read live operational JSON responses.
- **Zero Drift:** Schema declarations sync with code changes, eliminating outdated documentation headaches.

10. QUALITY ASSURANCE & TESTING STRATEGY

To ensure system stability under heavy transactional scenarios, a comprehensive quality assurance layout is enforced across backend and frontend boundaries:

- **Manual API Testing using Swagger:** Provides structured visual exploration and immediate validation of active request/response contracts during localized endpoint adjustments.
- **Postman Collection Validation:** Ensures rapid integration testing, environment variable parity, and bulk request staging configurations.
- **Load Testing with Autocannon:** Benchmarks server resilience, requests-per-second thresholds, and continuous database connection pooling stamina.
- **Frontend Performance using Lighthouse:** Audits core web vitals, page load dynamics, semantic structure accessibility, and bundle delivery footprints.

11. BACKEND LOAD TESTING & STRESS PERFORMANCE METRICS

To verify the production readiness of the backend API, the application underwent rigorous stress testing using Autocannon. The load testing script bombarded the Express runtime with continuous concurrent pipeline connections for a duration of 30 seconds.

LOAD TESTING METRIC	RESULT VALUE	ARCHITECTURAL ASSESSMENT
Average Response Time	35 ms	ULTRA-LOW LATENCY
Requests per Second (RPS)	900+ req/sec	HIGH THROUGHPUT CAPACITY
Data Throughput	1.8 MB/s	OPTIMIZED PAYLOAD TRANSFER
Failed Requests Count	0 failures	100% STABILITY UNDER LOAD
HTTP Keep-Alive	Enabled	Optimizes connection re-use patterns efficiently.
Test Execution Window	30 Seconds	Sustained continuous heavy concurrency matrix.

Performance Highlights Breakdown

- **Zero Failed Requests:** The Express middleware pipeline processed over 27,000 requests without a single connection drop or execution timeout.
- **Database Sub-Millisecond Speed:** Leveraged connection pooling via Prisma ORM to prevent database connection exhaustion under heavy loads.
- **Optimized Middleware Stack:** Removed redundant logic to ensure fast request-response lifecycles.

12. FRONTEND QUALITY & GOOGLE LIGHTHOUSE AUDIT

The client application was thoroughly audited using Google Lighthouse inside a simulated mobile environment to ensure the user experience remains fast and fluid even on lower-tier networks.

LIGHTHOUSE CATEGORY	AUDIT SCORE	REAL-WORLD UX ASSESSMENT STATUS
Performance	92 / 100	EXCELLENT
Accessibility	91 / 100	EXCELLENT
Best Practices	96 / 100	EXCELLENT
SEO Optimization	92 / 100	EXCELLENT

13. CORE WEB VITALS DEEP-DIVE ANALYSIS

Beyond broad categories, the React frontend was tuned to pass Google's strict Core Web Vitals criteria on mobile network topologies.

CORE WEB VITAL METRIC	MEASURED RESULT	PERFORMANCE THRESHOLD ASSESSMENT
First Contentful Paint (FCP)	2.4 s	GOOD
Largest Contentful Paint (LCP)	2.9 s	GOOD
Total Blocking Time (TBT)	40 ms	EXCELLENT
Cumulative Layout Shift (CLS)	0.00	PERFECT-ZERO SHIFT
Speed Index	2.4 s	EXCELLENT

Frontend Structural Optimization Techniques

- **Production Bundling via Vite:** Splitting dependencies into optimized chunks, reducing initial JavaScript payloads.
- **Code Splitting & Lazy Loading:** Components render only when required by the client routing path.
- **Asset Management:** Strict image dimensions and modern formats prevent unexpected layout shifting during page assembly.

14. SECURITY ARCHITECTURE & THREAT MITIGATION

The system is built on a zero-trust model, applying defense-in-depth across the data storage and web layers.

SECURITY FEATURE	IMPLEMENTATION STRATEGY	MITIGATED VULNERABILITY VECTOR
Password Cryptography	BcryptJS with high adaptive salt routing	Protects user credentials against brute-force and rainbow table attacks.
JWT Token Security	Stateless tokens signed via HS256 algorithm	Prevents session tampering and spoofing attempts.
HTTP Security Headers	Helmet.js integration middleware	Mitigates Cross-Site Scripting (XXSS) and clickjacking vectors.
Rate Limiting	Express Rate Limit (100 reqs / 15 mins / IP)	Thwarts Denial of Service (DoS) and brute-force login attempts.
Input Sanitization	Strict Joi schema verification layers	Blocks malicious schema injections before processing.
CORS Policy	Explicit origin whitelisting constraints	Prevents unauthorized cross-origin data extraction.

15. CLOUD DEPLOYMENT & DEVOPS ARCHITECTURE

The system leverages a modern cloud native topology, decoupling the application into isolated, managed hosting environments for maximum reliability.

Cloud Infrastructure Ecosystem

- **Frontend Application Layer:** Hosted via Vercel, pushing static builds onto a high-performance Global Edge Network.
- **Backend API Application Layer:** Deployed on Railway inside an isolated Docker container, using automated builds connected to git pushes.
- **Relational Database Layer:** Maintained via Neon PostgreSQL, utilizing serverless architecture to deliver automatic scaling and fast query execution.
- **CI/CD Pipelines:** Standardized with GitHub, triggering automated build and deployment previews on every code check-in.

Production Docker Configuration

```
FROM node:18-alpine
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npx prisma generate
EXPOSE 8080
CMD ["node", "src/server.js"]
```

16. SYSTEM METRICS & PROJECT STATISTICS

ARCHITECTURAL ATTRIBUTE	MEASURED SYSTEM VALUE
System Architecture Topology	Decoupled Full-Stack Architecture
Frontend Layer Framework	React JS backed by Vite Build Tooling
Backend API Server Engine	Node.js utilizing Express Framework
Database Cluster Engine	Neon Serverless PostgreSQL
Data Mapping Layer	Prisma Object-Relational Mapper (ORM)
Authentication Scheme	Stateless cryptographically signed JWT
Active Human System Roles	3 Isolated Tiers (Admin, Resident, Staff)
Production REST Endpoints	40+ Fully Documented Structural Paths
Database Schema Relations	9 Normalized Tables
Core Business Logic Modules	10+ Encapsulated Functional Folders

17. PROJECT FOLDER BLUEPRINT & MODULAR DESIGN

```
src/  
  app.js           # Express application configuration & setup  
  server.js        # Server entry point & connection listener  
  config/          # Environment configurations & database instances  
  middlewares/     # JWT Auth, RBAC guards, Joi validation layers  
  modules/         # Encapsulated domain business modules  
    auth/          # Auth routers, controllers, and validation rules  
    buildings/     # Physical infrastructure governance logic  
    apartments/    # Unit vacancy maps & assignments  
    residents/     # Resident profiles & soft delete pipelines  
    staff/         # Operator roles & shift tasks  
    documents/     # Multer pipeline & Cloudinary bridges  
    requests/      # Maintenance tickets & state flows  
    visitors/      # Access logs & visitor codes  
    announcements/ # Global notice distribution  
    dashboard/     # Administrative data aggregations  
  prisma/          # Schema declarations & database seeding scripts  
  utils/           # AppError handlers & email engines
```

18. STRATEGIC FUTURE ROADMAP & SYSTEM SCALE

- 1. High-Performance Caching Layer:** Integrating a Redis cluster to cache slower database results, such as building analytics and tenant records, reducing database read loads.

2. **Real-Time Bidirectional Event Streaming:** Upgrading the notification system with Socket.io to push instant browser alerts to security staff when residents authorize new visitor entry codes.
3. **Automated Biometric Gate Integration:** Exposing specialized, high-security endpoints to let physical QR scanners at compound gates validate visitor records instantly without manual staff actions.

GITHUB REPOSITORIES & LIVE ENVIRONMENTS

- **Interactive Swagger API Documentation:** [Swagger Live Link](#)
- **Frontend Application Link (Vercel Production):** [Live Web App](#)
- **GitHub Source Repository:** Available via authenticated engineering team handles.

Thank You

Designed and Engineered with precision for modern community infrastructure.